

Bypass VOL Connector Benchmarking Results

The Bypass VOL connector (https://github.com/LifeboatLLC/MT-HDF5/tree/main/vol_bypass) serves as an important validation platform for the multi-threaded HDF5 implementation. Benchmarking and scalability studies help identify several lock-ordering and synchronization issues that were subsequently resolved in H5I and related packages. See https://github.com/LifeboatLLC/MT-HDF5/tree/main/vol_bypass/test for benchmark scripts.

Performance measurements on a Linux system with 14 CPU cores demonstrated substantial I/O improvements for both contiguous and chunked datasets. Depending on the application configuration and the number of threads in the connector's internal thread pool, we observed throughput improvements of up to 4.5× relative to the baseline single-threaded HDF5 implementation, with peak performance approaching 5 GB/s for large datasets. Additional experiments involving concurrent access to multiple datasets demonstrated consistent speedups of approximately 2.5× when using multi-threaded applications. These results confirm the feasibility of introducing meaningful multi-threaded concurrency into the HDF5 software stack while preserving compatibility with the existing HDF5 architecture. See Figures 1-6.

To evaluate the practical applicability of the proposed architecture on leadership-class computing systems, we conducted performance studies on the NERSC Perlmutter supercomputer. The benchmarks were performed using a conventional single-threaded application that accessed HDF5 data through the Bypass VOL connector, which internally utilizes a thread pool to execute concurrent I/O operations. For a 64 GB chunked dataset, throughput increased from 5.4 GB/s to 22.7 GB/s as the internal thread pool size increased from one to sixteen threads. These results demonstrate that substantial performance improvements can be achieved on production HPC systems without requiring application-level parallelization or modifications to existing HDF5 programs. See Figure 7.

Performance on Bypass VOL connector for multi-threaded application when reading chunked dataset

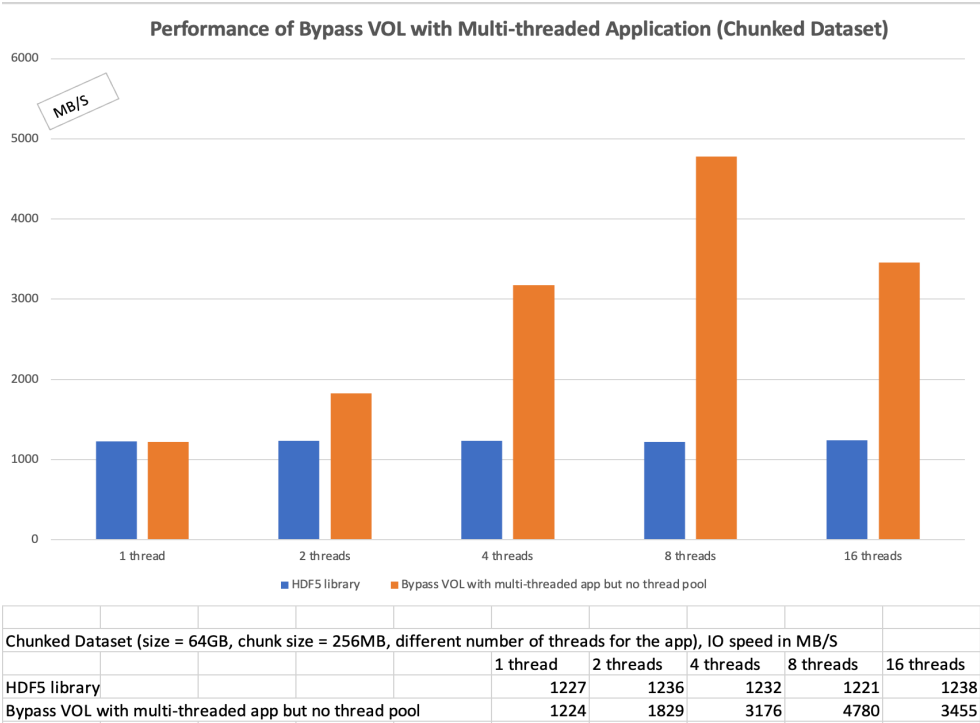


Figure 1: The HDF5 application reads a 64 GB chunked dataset, with the workload evenly distributed across application threads. The application scales effectively, achieving approximately 5 GB/s I/O throughput with eight threads.

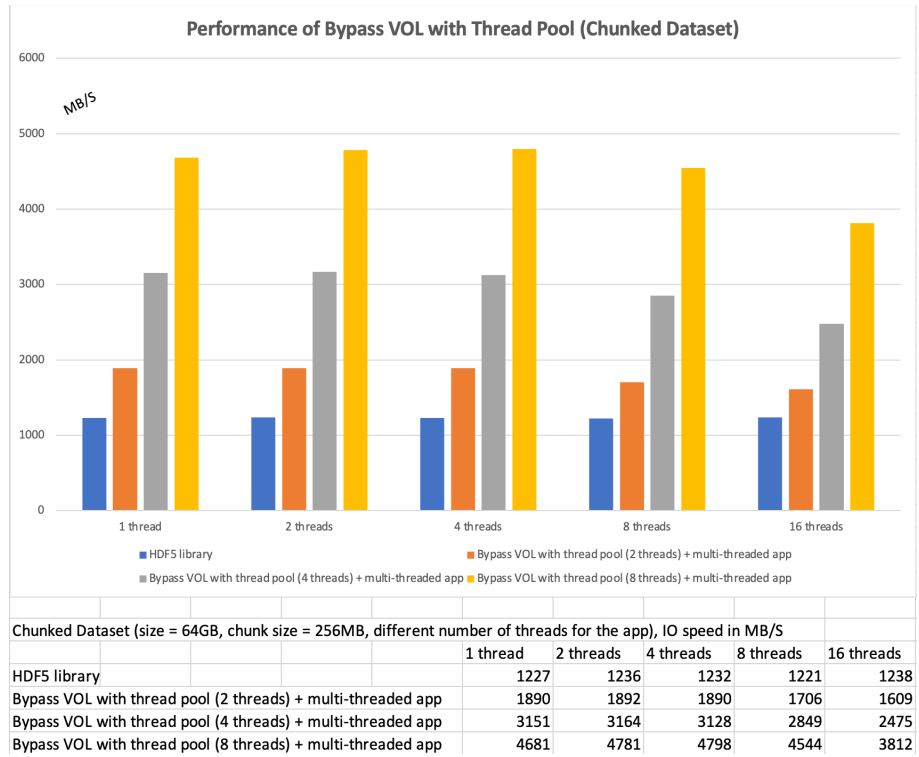


Figure 2: The HDF5 application reads a 64 GB chunked dataset using 1 to 16 application threads, with the workload evenly distributed across threads. The orange, gray, and yellow bars represent thread pool sizes of 2, 4, and 8 threads, respectively. The best performance is achieved when using a thread pool with eight threads.

Performance on Bypass VOL connector for multi-threaded application when reading contiguous dataset

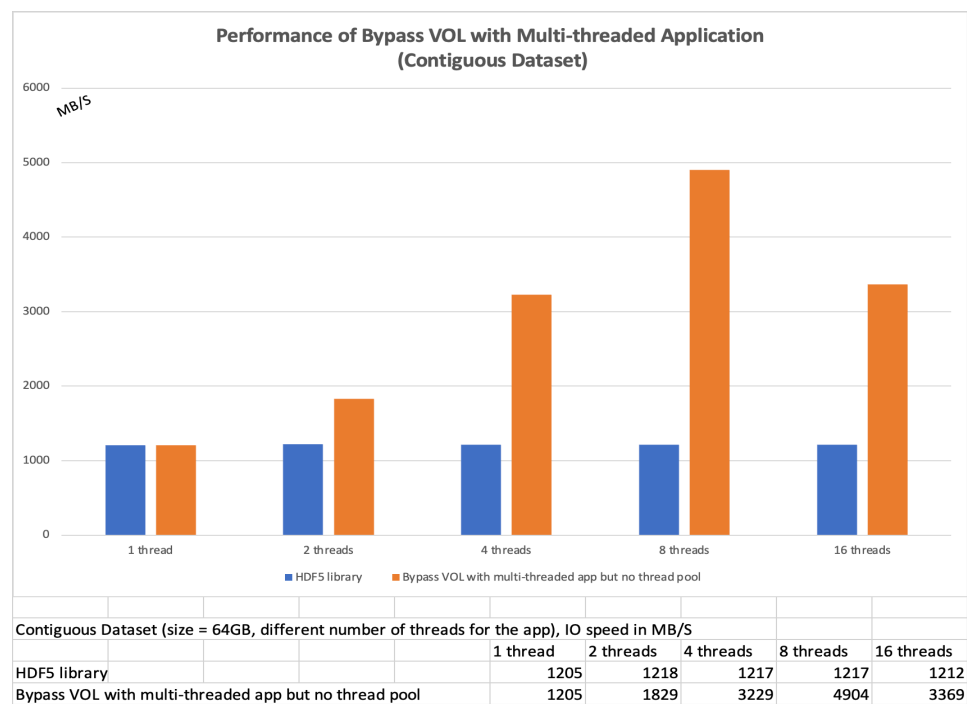


Figure 3: The HDF5 application reads a 64 GB contiguous dataset, with the workload evenly distributed across application threads. The application scales effectively, achieving approximately 5 GB/s I/O throughput with eight threads. We observe the same performance trend as shown in Figure 1 for the chunked dataset.

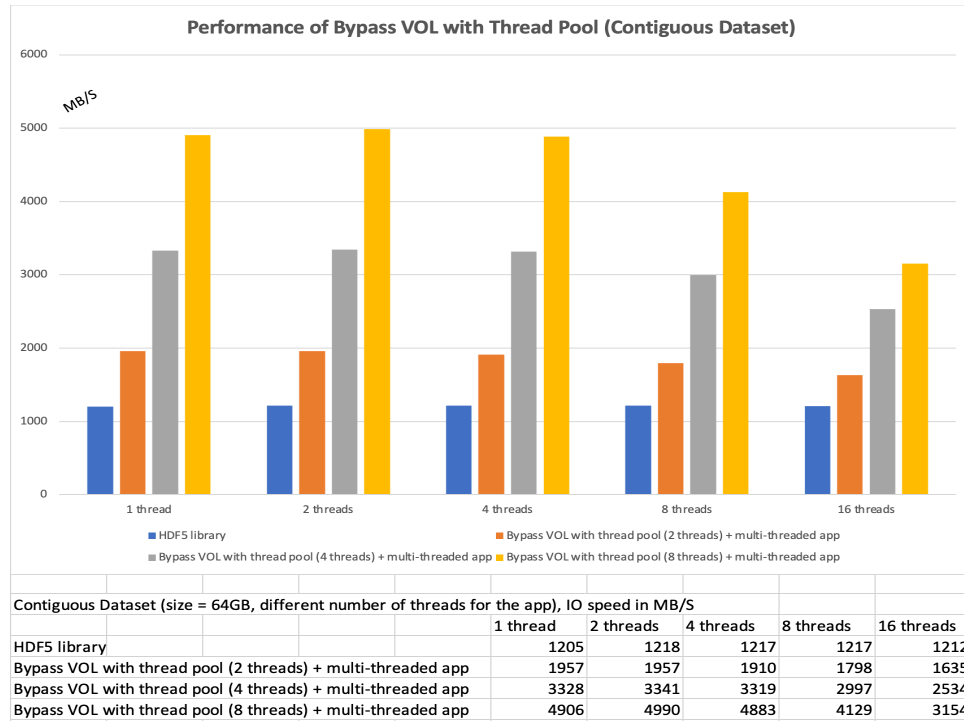


Figure 4: The HDF5 application reads a 64 GB contiguous dataset using 1 to 16 application threads, with the workload evenly distributed across threads. The orange, gray, and yellow bars represent thread pool sizes of 2, 4, and 8 threads, respectively. The best performance is achieved when using a thread pool with eight threads, consistent with the results observed for chunked dataset (see Figure 2).

Performance on Bypass VOL connector for reading multiple contiguous datasets

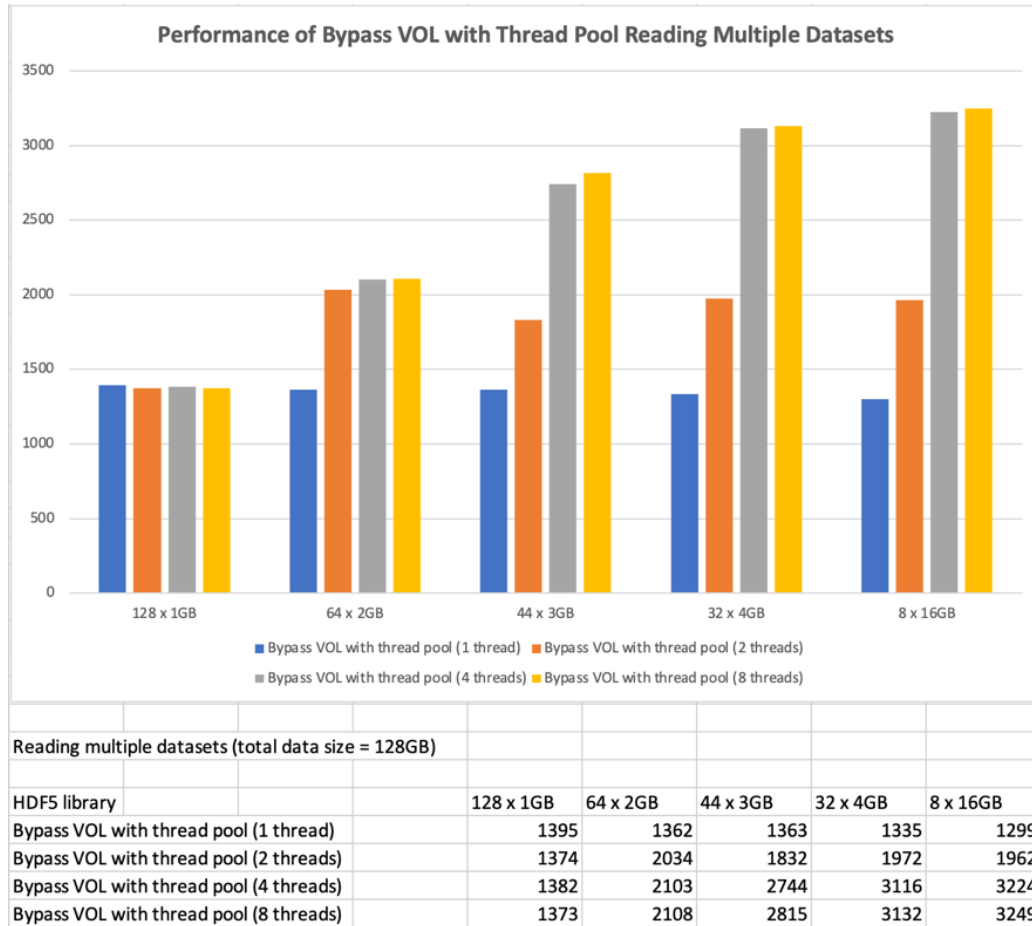


Figure 5: The single-threaded HDF5 application reads multiple contiguous datasets with a total size of 128 GB using thread pools of varying sizes. The blue, orange, gray, and yellow bars represent thread pool sizes of 1, 2, 4, and 8 threads, respectively. We observe a performance difference when opening 128 1GB datasets compared to opening 32 or 8 datasets, with the larger number of smaller datasets resulting in reduced overall performance.

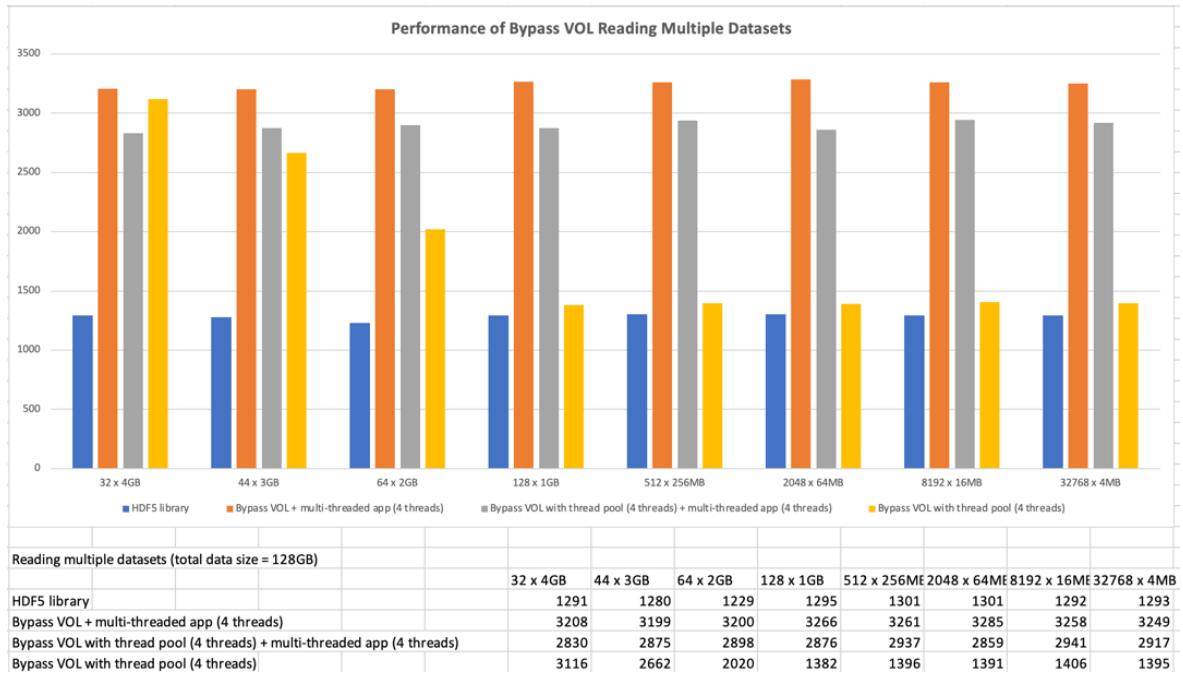


Figure 6: The HDF5 single- and multi-threaded applications read multiple datasets, with the workload evenly distributed across threads in the multi-threaded case. A consistent 2.5 $\times$ speedup is achieved across dataset counts ranging from 32 to 32,768 when the application uses four threads and no thread pool in the VOL connector. Adding a thread pool to the VOL connector does not improve performance for the multi-threaded application, as indicated by the gray bars. The performance of the VOL connector with a single-threaded application (yellow bars) drops significantly for datasets smaller than 2 GB. We will investigate the cause of this behavior.

Performance on Bypass VOL connector with thread-pool on the NERSC Perlmutter supercomputer

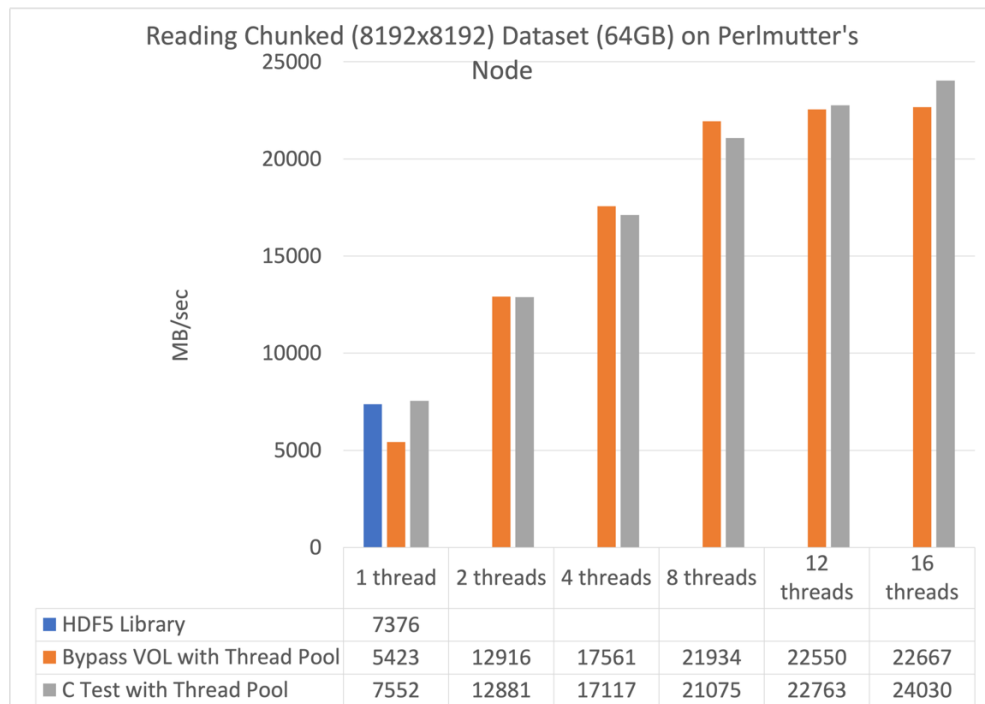


Figure 7: **Performance of the Bypass VOL connector on NERSC Perlmutter.** Throughput measured for a 64 GB chunked dataset using 1 MB I/O requests. The Bypass VOL connector scales with increasing thread count and reaches approximately 22.7 GB/s at sixteen threads, compared with approximately 7.3 GB/s for the native HDF5 library.